

# Trabajo Práctico 8

## Inciso 1

---

Una sentencia puede ser simple o compuesta, ¿Cuál es la diferencia?

### Respuesta

La diferencia fundamental entre una sentencia (también llamada enunciado) simple y una compuesta radica en su **estructura y capacidad de agrupación**:

- **Sentencia Simple:** Se considera el bloque de construcción más básico de los lenguajes imperativos. El ejemplo más elemental es la **sentencia de asignación**, la cual toma el r-valor de una expresión y lo asigna a un l-valor, modificando así el estado de la memoria. Estas instrucciones suelen ejecutarse en un orden específico, habitualmente de arriba hacia abajo, sin desviaciones.
- **Sentencia Compuesta (o Bloque):** Consiste en una **secuencia de sentencias** que se ejecutan secuencialmente, comenzando por la primera y terminando en la última. Su función principal es permitir que un conjunto de instrucciones sea tratado como una **única unidad sintáctica** en lugares donde el lenguaje solo esperaría una sentencia simple.

### Características distintivas de las sentencias compuestas:

- **Delimitadores sintácticos:** Dependiendo del lenguaje, se utilizan marcas específicas para agrupar estas secuencias. Por ejemplo, en Pascal y Ada se usan las palabras reservadas `begin` y `end`, mientras que en C, Java y C++ se utilizan llaves `{ }`.
- **Gestión de Ambientes:** Una sentencia compuesta puede incluir sus propias **declaraciones locales**. Estas variables internas pueden "enmascarar" a variables externas si comparten el mismo nombre, limitando su alcance al interior del bloque.
- **Jerarquía:** Es posible incluir sentencias compuestas dentro de otras para construir bloques de código cada vez más grandes y complejos.
- **Impacto en el flujo:** Se utilizan comúnmente dentro de estructuras de control, como el `if` o el `while`, para ejecutar múltiples acciones cuando

se cumple una condición.

## Inciso 2

---

Analice como C implementa la asignación.

### Respuesta

En el lenguaje C, la **asignación** se implementa no solo como una sentencia básica, sino fundamentalmente como una **expresión con efectos colaterales**, lo que le otorga características particulares en comparación con otros lenguajes imperativos.

#### 1. Semántica Operacional

Desde la perspectiva de la semántica operacional, la asignación en C realiza una **modificación destructiva** del valor contenido en una celda de memoria

#### 2. Asignación como expresión

A diferencia de otros lenguajes donde la asignación es meramente una sentencia, en C se define como una **expresión**. Esto implica que:

- **Devuelve un valor:** Una operación de asignación devuelve el mismo valor que ha sido asignado.
- **Precedencia de derecha a izquierda:** Debido a que devuelve un valor, permite el **encadenamiento de asignaciones** (por ejemplo, `a = b = c = 0`), donde primero se asigna el valor de `c` a `b`, y ese resultado se asigna luego a `a`.
- **Uso en estructuras de control:** Es posible utilizar una asignación dentro de una condición (ej. `if (a = getchar())`), lo cual es válido sintácticamente porque la asignación produce un valor que el `if` puede evaluar.

#### 3. Efectos colaterales

En C, la modificación del estado de la memoria (el cambio del valor en la celda) se considera técnicamente un **efecto colateral** de la evaluación de la expresión de asignación. Mientras que la función principal de una expresión suele ser devolver un valor sin alterar la memoria, en C los operadores de

asignación (y otros como el incremento `++`) están diseñados específicamente para provocar este cambio de estado.

#### 4. Ligadura Dinámica

La asignación es el mecanismo mediante el cual se produce la **ligadura dinámica (binding)** entre una variable y su valor. Aunque el nombre y el tipo de la variable en C suelen ligarse de forma estática (en compilación), el **R-valor** se liga en tiempo de ejecución y puede ser modificado repetidamente mediante sentencias de asignación.

### Inciso 3

---

¿Una expresión de asignación puede producir efectos laterales que afecten al resultado final, dependiendo de cómo se evalúe? De ejemplos.

#### Respuesta

Sí, una **expresión de asignación puede producir efectos laterales** (también llamados efectos colaterales) que afecten el resultado final del programa, dependiendo de cómo el lenguaje evalúe dicha expresión y el contexto en el que se utilice.

En lenguajes como C, C++ y Java, la asignación no es solo una sentencia, sino una **expresión que devuelve un valor**. El hecho de modificar el estado de la memoria (cambiar el valor de una variable) mientras se genera un resultado para ser usado en una expresión mayor es, técnicamente, un efecto lateral. Ejemplos:

#### 1. Asignaciones dentro de estructuras de control

En C, es posible realizar una asignación dentro de la condición de un `if` o un `while`. El efecto lateral es la modificación de la variable, pero el resultado final de la expresión determina si el flujo continúa.

- **Ejemplo:** `if (a = getChar())`.
  - **Efecto lateral:** La variable `a` cambia su valor al recibir el carácter.
  - **Resultado final:** El `if` evalúa el valor asignado a `a`. Si es distinto de cero (verdadero en C), la condición se cumple.
- **Riesgo de error:** Un error común es escribir `while(a = NULL)` (asignación) en lugar de `while(a == NULL)` (comparación). En el primer

caso, el efecto lateral es poner `a` en `NULL`, y la expresión siempre resultará en "falso", impidiendo que el bucle se ejecute, independientemente del valor previo de `a`.

## 2. Asignaciones múltiples y encadenadas

Debido a que la asignación devuelve el valor asignado, se pueden encadenar varias en una sola línea. La semántica de estos lenguajes establece una **precedencia de derecha a izquierda**.

- **Ejemplo:** `a = b = c = 0;`
  - Aquí, el efecto lateral de asignar `0` a `c` produce un resultado (`0`) que luego se usa para asignar a `b`, y así sucesivamente hasta llegar a `a`.

## 3. Interacción con la Evaluación en Circuito Corto

El resultado final de una expresión puede variar si una asignación con efecto lateral se encuentra en una parte del código que el compilador decide **no evaluar** por optimización.

- **Ejemplo:** `if (condicion || (a = 10))`.
  - Si `condicion` es verdadera, el lenguaje utiliza **evaluación en cortocircuito** y no evalúa el segundo operando.
  - **Consecuencia:** El efecto lateral (asignar 10 a `a`) **nunca ocurre**, por lo que el valor de `a` al finalizar la sentencia dependerá totalmente de si la primera condición fue suficiente para determinar el resultado.

## Inciso 4

---

Qué significa que un lenguaje utilice circuito corto o circuito largo para la evaluación de una expresión. De un ejemplo en el cual por un circuito de error y por el otro no.

### Respuesta

La diferencia entre utilizar un esquema de evaluación de **circuito corto** (o evaluación perezosa) y uno de **circuito largo** radica en si el sistema evalúa todos los operandos de una expresión booleana o si se detiene apenas el resultado final está determinado.

### Circuito corto

Se refiere a la técnica de evaluación de expresiones booleanas (como **AND** y **OR**) donde los operandos se procesan de izquierda a derecha, pero **la evaluación finaliza en el momento en que se encuentra un operando que determina el resultado de toda la expresión.**

- **Lógica del AND:** Si el primer operando es **falso**, la expresión completa será falsa sin importar el valor del segundo. Por lo tanto, el sistema no evalúa el resto,.
- **Lógica del OR:** Si el primer operando es **verdadero**, la expresión completa será verdadera y no es necesario continuar evaluando.

## Circuito largo

Aunque las fuentes no utilizan explícitamente el término "circuito largo", se refieren a este esquema como la **evaluación completa o estricta**, donde el lenguaje obliga a evaluar todos los componentes de la expresión antes de determinar su valor de verdad.

## Ejemplo

Prevención de Errores vs. Fallo del Programa

**Caso: Acceso a un arreglo (Array)** Imagina la siguiente expresión: `(i < a.length) AND (a[i] == 0)`.

- **Con Circuito Corto:** Si el índice `i` es igual o mayor al tamaño del arreglo (`a.length`), la primera parte es **falsa**. El lenguaje termina la evaluación ahí mismo y **evita acceder al arreglo**, por lo que el programa continúa funcionando correctamente sin errores de desbordamiento.
- **Con Circuito Largo:** El lenguaje intentaría evaluar ambos lados. Al tratar de procesar `a[i]` con un índice fuera de rango, se produciría un **error de ejecución (crash)** que detendría el programa, a pesar de que la primera condición ya indicaba que el acceso no era seguro.

## Inciso 5

---

¿Qué regla define Delphi, Ada y C para la asociación del else con el if correspondiente? ¿Cómo lo maneja Python?

**Respuesta**

Para resolver el problema de ambigüedad conocido como "**Dangling Else**" (else suelto), los lenguajes que mencionas aplican distintas reglas sintácticas para determinar a qué `if` pertenece un `else` en estructuras anidadas:

### 1. Delphi (Pascal) y C: Regla de Proximidad

Ambos lenguajes resuelven la ambigüedad mediante la **Regla de Proximidad**.

- **Funcionamiento:** Bajo esta regla, el `else` se empareja automáticamente con el `if` **más cercano** que se encuentre abierto en la estructura jerárquica.
- **Nota sobre Delphi:** Aunque Delphi no aparece mencionado explícitamente por ese nombre en todos los apartados, las fuentes clasifican a **Pascal** (lenguaje en el que se basa Delphi) dentro de este grupo junto con C.

### 2. Ada: Cierres Explícitos

Ada utiliza una estrategia diferente basada en **Cierres Explícitos** para delimitar el alcance de la estructura.

- **Funcionamiento:** En lugar de depender de la cercanía, Ada obliga al uso de una palabra clave de cierre, específicamente **end if**. Esto elimina cualquier ambigüedad, ya que el programador define exactamente dónde termina cada bloque condicional.

### 3. Python: Indentación Obligatoria

Python rompe con los esquemas anteriores al utilizar la estructura visual del código como regla sintáctica.

- **Funcionamiento:** Utiliza la **indentación obligatoria** para definir los bloques de código. En este caso, el `else` se asociará con el `if` que esté en su mismo nivel de sangría (espaciado).
- **Palabra clave adicional:** Python también introduce la palabra clave **elif** para manejar selecciones múltiples de forma más clara dentro de su esquema de indentación.

## Inciso 6

---

¿Cuál es la construcción para expresar múltiples selección que implementa C?  
¿Trabaja de la misma manera que la de Pascal, ADA o Python?

## Respuesta:

En el lenguaje **C**, la construcción para expresar la selección múltiple es la sentencia **switch**, la cual utiliza etiquetas constantes para dirigir el flujo del programa.

Sin embargo, esta construcción **no trabaja de la misma manera** que en Pascal, Ada o Python, ya que presenta diferencias críticas en su semántica y reglas de seguridad:

### 1. El fenómeno del "Falling Through" en C

A diferencia de los otros lenguajes mencionados, en **C** (y **C++**), si se omite la sentencia **break** al final de un bloque **case**, la ejecución **continúa automáticamente** en el siguiente caso,. Esto es una característica única que permite agrupar varios casos bajo una misma acción, pero también es una fuente común de errores si el programador olvida el **break**.

### 2. Comparación con Pascal y Ada

- **Pascal:** Utiliza la estructura **case-of-else-end**,. Las fuentes señalan que es "insegura" si no se incluye la cláusula **else** para cubrir valores no especificados.
- **Ada:** Implementa reglas mucho más **estrictas** que C. En Ada, el programador **debe contemplar todos los valores posibles** del tipo de dato en los casos, o bien utilizar de forma obligatoria la palabra clave **others** para manejar el resto de las opciones, garantizando mayor confiabilidad,.

### 3. Comparación con Python

**Python** no utiliza una estructura de **switch** tradicional similar a la de C. En su lugar:

- Maneja la selección múltiple mediante el uso de la palabra clave **elif**.
- Utiliza la **indentación obligatoria** para delimitar los bloques de cada opción, eliminando la ambigüedad sintáctica sin necesidad de cierres explícitos como el **break** de C o el **end if** de Ada.

En resumen, mientras que **C** prioriza la flexibilidad con sus etiquetas constantes y el *falling through*, lenguajes como **Ada** priorizan la seguridad

mediante la cobertura total de casos, y **Python** se enfoca en la legibilidad a través de su estructura de bloques indentados.